



微算機系統 2026 MICROPROCESSOR SYSTEMS

Instructor : Yen-Lin Chen(陳彥霖)

Dept. Computer Science and Information Engineering

National Taipei University of Technology

ylchen@ntut.edu.tw



實驗三：

BCD加法器 電路設計

組合電路設計流程 & 階層化設計



繳交規定

- 作業繳交項目：(P.4, P.5有詳細規定)
 - 小組專案報告
 - 各小組指派一位同學上傳就好
 - 以 pdf 格式繳交，格式錯誤等同未繳交
 - 個人報告
 - 每位同學都一定要繳交
 - 以 pdf 格式繳交，格式錯誤等同未繳交
- 專案驗收的期限為4/1(三)課程結束前
- 小組及個人報告的繳交期限為 4/3(五)的23:59前



小組專案內容

- 目標一 用一個資料夾放，內有
 - 專案名.qsf
 - 專案名.qpf
 - 專案名.vhd(硬碟映像檔)
 - db(資料夾)
- 專案報告(pdf格式)
 - 封面：第幾次實驗、組別、班級學號姓名、日期
 - 實驗內容
 - 實驗過程及結果
 - 程式碼



個人報告內容

- 心得 (每個組員最少150字、中英文皆可)
- 組員貢獻比例及工作內容(組員%數加總必須等於100%)



繳交規定

- 小組專案報告與個人報告繳交期限：**4/3(五)的23:59**以前上傳至北科i學園PLUS→作業
- 實驗、報告遲交一週內打8折，第二週打6折
- 之後不接受補交報告
- 繳交方法：請同學上傳作業時將資料夾壓縮成一個zip上傳到北科i學園PLUS。Zip的檔名為“Lab03_第XX組.zip” (XX為組別號碼)，資料夾內需含有實驗專案及實驗報告(pdf檔)。



配分方式

- 實驗驗收70 % (實驗目標— 70%)
- 小組報告 20%
- 個人報告 10%
- 報告扣分標準
 - 封面內容(學號、姓名)：扣3分
 - 缺少實驗內容、程式碼、心得、組員貢獻比例：各扣4.5分
 - 缺少實驗過程及結果：扣9分
 - 實驗過程及結果和心得不完整會依照狀況扣分，最高扣到該項目的上限分數
- 實驗專案不完整或抄襲者會依照狀況扣分，**最高扣70分**。



Package與Component教學

- 第一步：建立邏輯電路
 - 將需要用到的輸入輸出做定義。
 - 建立需要實現的邏輯電路。
 - 以 fulladd.vhd 為範例，此步驟需要建立一個獨立檔，內容如下。

```
1  library ieee;  
2  use ieee.std_logic_1164.all;  
3  
4  entity fulladd is  
5  port( cin,x,y : in std_logic;  
6       s,Cout : out std_logic);  
7  end fulladd;  
8  
9  architecture func of fulladd is  
10 begin  
11     s <= x xor y xor cin;  
12     Cout <= (x and y) or (cin and x) or (cin and y);  
13 end func;
```



Package與Component教學

- 第二步：建立 Component
 - 將定義好的 PORT 部分填入 Component 中。
 - 此時的 Component 會是一個類似 Class 的存在，具有變數定義與函式功能。
 - 在主程式中使用相當於建立一個 Class object，用“port map”來呼叫 Component 中要使用的函式。
 - 可以執行 Component 底下的邏輯電路功能。

```
component fulladd
  port( cin,x,y : in std_logic;
        s, Cout : out std_logic);
end component;
```



Package與Component教學

- 第三步：建立 Package
 - 將 Component 包成 Package。
 - Package 建立完成後會是一個類似 C++ 中 “.h” 檔的存在。
 - 建立完後要使用 Use 來匯入 Package，使得主程式可以使用在 Package 中定義好的函式(邏輯電路)。
 - 以 lab3_package.vhd 為範例，第二、三步驟各需要建立一個獨立檔，內容如下。

```
1  library ieee;  
2  use ieee.std_logic_1164.all;  
3  
4  package LAB03_package is  
5  |  
6  | component fulladd  
7  |   port( Cin, x, y : in std_logic;  
8  |         s, Cout : out std_logic);  
9  | end component fulladd;  
10 |  
11 | component hex  
12 |   port( w, x, y, z : in std_logic;  
13 |         a, b, c, d, e, f, g : out std_logic);  
14 | end component hex;  
15 |  
16 end LAB03_package;
```



Package與Component教學

- 第四步：將 Package 匯入主程式
 - 使用 Quartus Prime 的 “Use work.Package名稱” 語法來匯入 Package。
 - 這裡的 “Use” 語法可以想成是 C++ 中的 “include”。
 - 匯入完成後就可以使用定義好的邏輯電路函式。

```
library ieee;
use ieee.std_logic_1164.all;
use work.LAB03_package.all;

entity LAB03 is
  port(
    A : in std_logic_vector(0 to 7);
    B : in std_logic_vector(0 to 7);
    a0, b0, c0, d0, e0, f0, g0 : out std_logic;
    a1, b1, c1, d1, e1, f1, g1 : out std_logic;
    overflow : out std_logic);
end LAB03;

architecture structure of LAB03 is
```



Package與Component教學

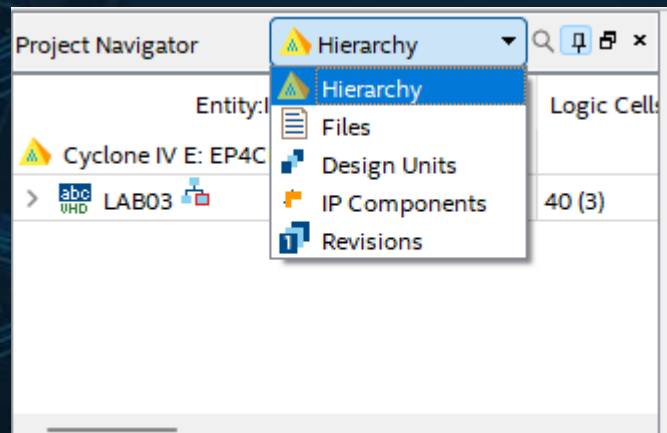
- 第五步：匯入主程式的函數使用
 - 先命名該步驟的 label (以圖中的 stage0: 為例)。
 - 若在使用相同 package 的情況下不定義 label 會被認為是重複定義，而出現 syntax error。
 - 語法使用“port map”將主程式定義的輸入對應到邏輯電路中的輸入。
 - 以 main.vhd 為範例，第四、五步驟要在主程式中執行，內容如下。

```
stage0: fulladd port map(A(0), B(0), '0', s(0), c(1));  
stage1: fulladd port map(A(1), B(1), c(1), s(1), c(2));  
stage2: fulladd port map(A(2), B(2), c(2), s(2), c(3));  
stage3: fulladd port map(A(3), B(3), c(3), s(3), c(4));
```



Top-Level 設定說明

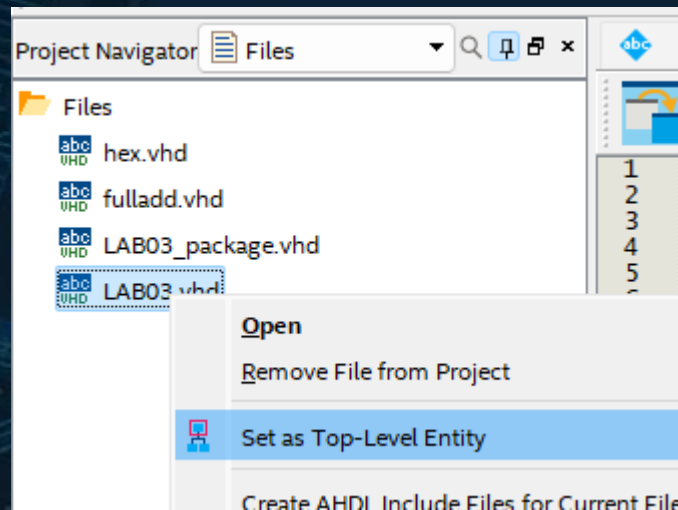
- 下拉式選單選取Files





Top-Level 設定說明

- 對專案使用的.vhd檔按滑鼠右鍵，選取”Set as Top-Level Entity”





Top-Level 設定說明

- 透過下拉選單回到Hierarchy，於編譯後即可看到使用於專案中的Component及其label

Entity:Instance	Logic Cells	Dedicated Logic Registers	I/O Reg
Cyclone IV E: EP4CE115F29C7			
LAB03	40 (3)	0 (0)	0 (0)
hex:display0	9 (9)	0 (0)	0 (0)
hex:display1	9 (9)	0 (0)	0 (0)
fulladd:stage0	1 (1)	0 (0)	0 (0)
fulladd:stage00			

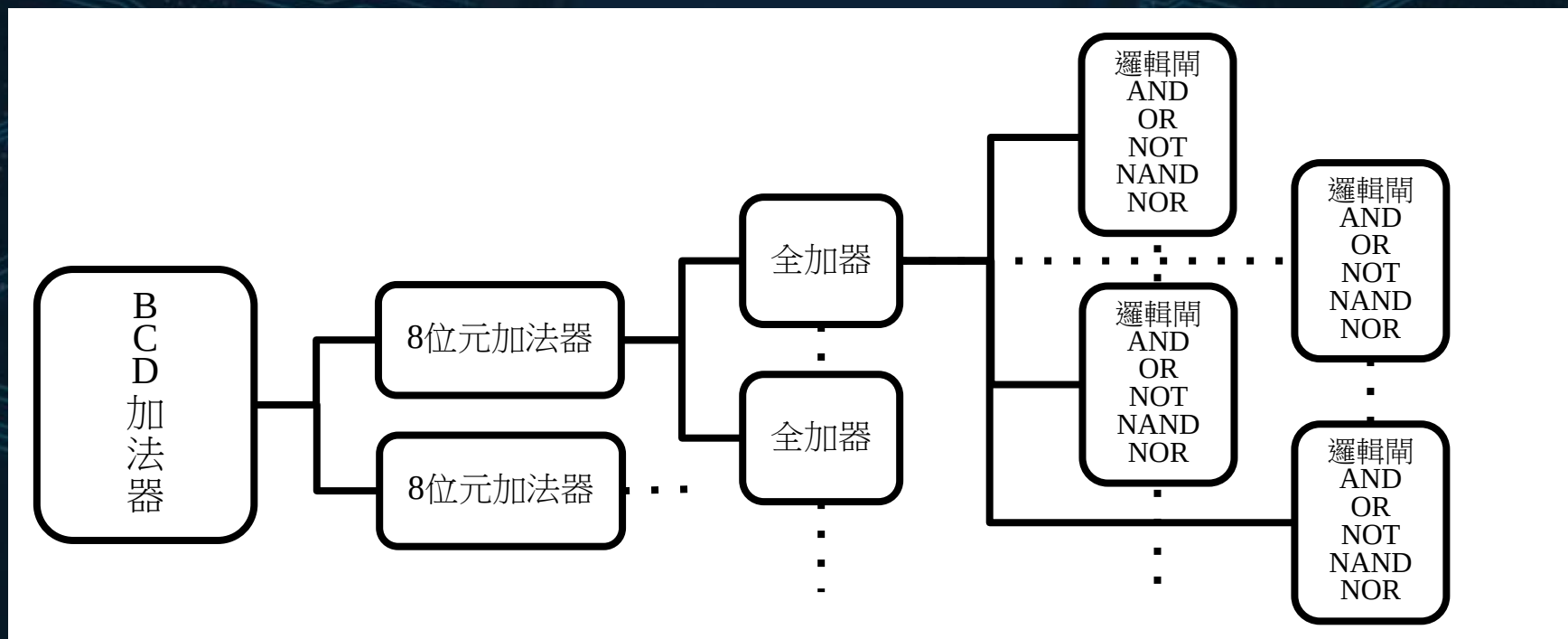


實驗目標 – BCD加法器



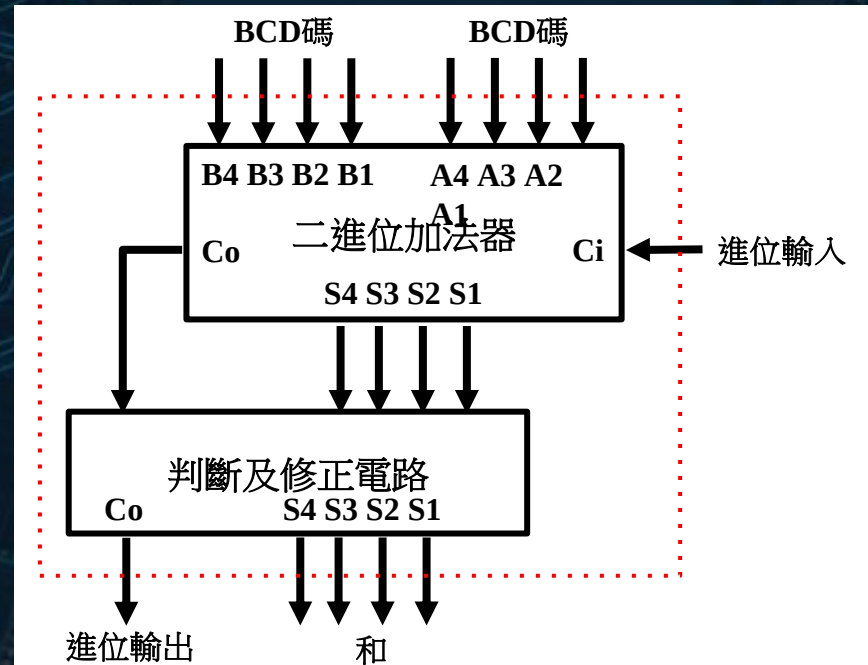
基本概念：階層化的設計概念

- 上層電路是由下層電路所合成，而本實習以實現一個 BCD 加法器，呈現電路階層化設計的概念，並以兩顆 7-Segment 七段顯示器顯示 0~99 相加之結果。
- 設計由上而下，實現電路由下而上。



BCD加法器之系統架構

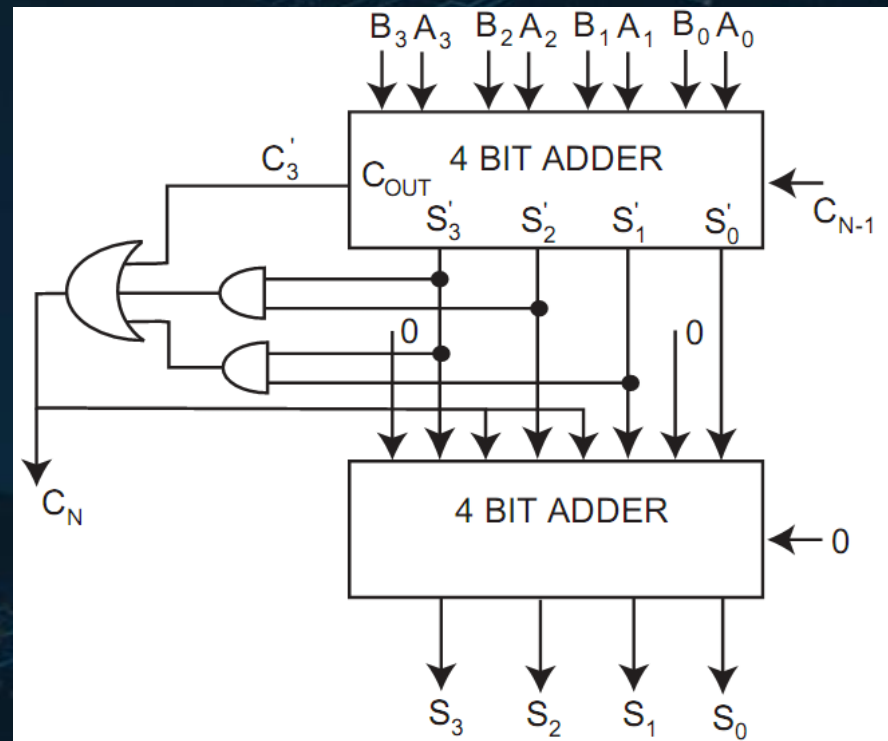
- A1-A4 與 B1-B4 相加若 S1-S4 是否超過 1001 (9) 時則必須由『判斷電路』判斷。
- 而超過 9 再利用『修正電路』作修正其『和』及產生『進位輸出』





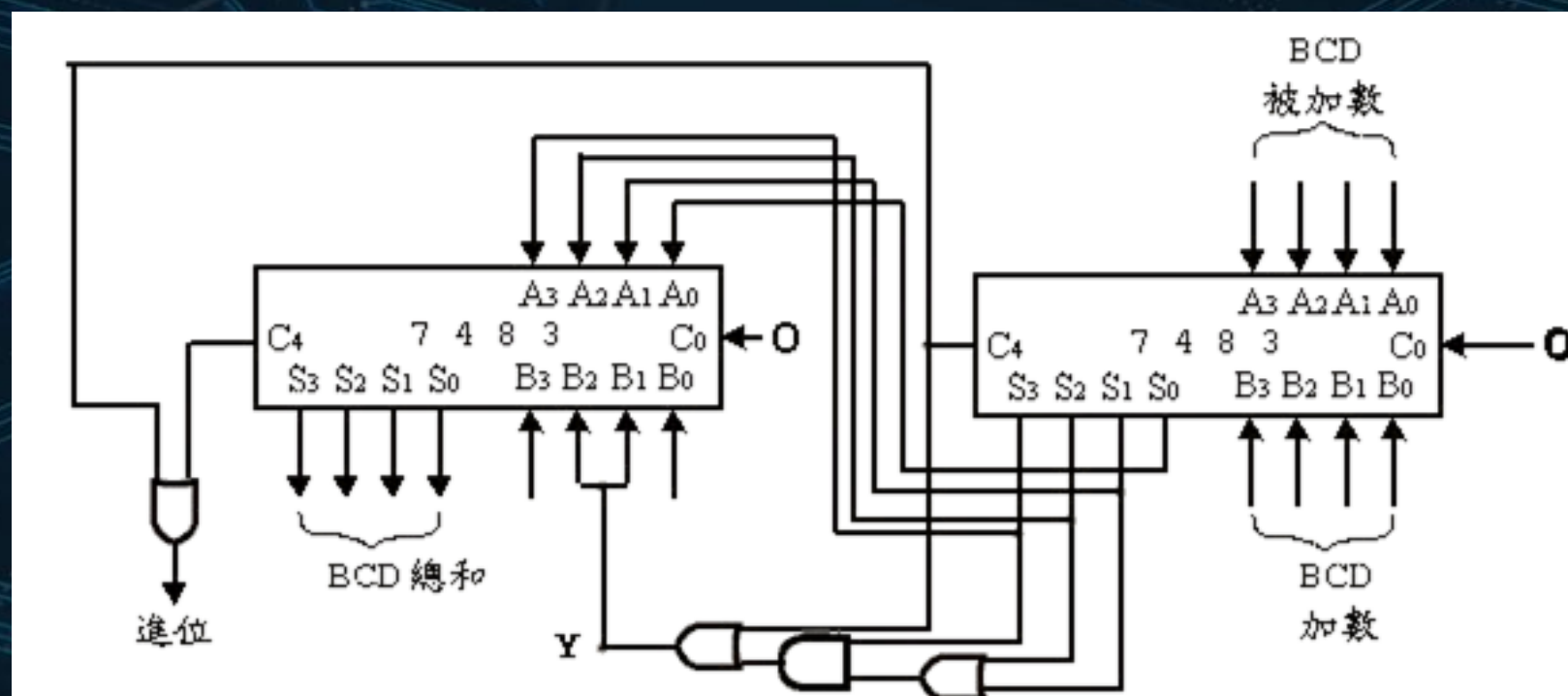
BCD加法器之電路

- 加法進位的條件：
 - $8 + 4 > 9 \Rightarrow$ 進位。
 - $8 + 2 > 9 \Rightarrow$ 進位。
- BCD加法器進位條件：
 - $S_3' \text{ AND } S_2'$ 是 1 就進位。
 - $S_3' \text{ AND } S_1'$ 是 1 就進位。
- CN 代表是否進位 (Carry)。
 - 若有進位 ($CN = 1$)，要在最終結果再加一個 0110 的值轉為 Decimal。





4bit - BCD加法器之電路





VHDL code 撰寫與實現

- 完成 BCD 加法器電路後，以階層化設計整合上次實驗一所實作之 7-Segment 七段顯示器電路之 VHDL 實作檔，再以 **Package 與 component 語法包裝** 並整合入 BCD 加法器電路中，而且在實驗板上面以點亮兩顆七段顯示器驗證 BCD 加法電路是否正確。



VHDL code

```
library ieee;
use ieee.std_logic_1164.all;
use work.LAB03_package.all;
...

entity LAB03 is
    port( A : in std_logic_vector(0 to 7);
          B : in std_logic_vector(0 to 7);
          a0, b0, c0, d0, e0, f0, g0 : out std_logic;
          a1, b1, c1, d1, e1, f1, g1 : out std_logic;
          overflow : out std_logic);
end LAB03;

architecture structure of LAB03 is
    signal c : std_logic_vector(1 to 8);
    ...
Begin
    ...
end structure;
```



包裝 Component 的範例

```
library ieee;  
use ieee.std_logic_1164.all;  
package LAB03_package is  
    component fulladd  
        port( Cin, x, y : in std_logic;  
             s, Cout : out std_logic);  
    end component fulladd;  
    component hex  
        port( w, x, y, z : in std_logic;  
             a, b, c, d, e, f, g : out std_logic);  
    end component hex;  
    ...  
end LAB03_package;
```



實驗目標: 8位元 BCD 加法器

1. 以1bit全加器為基礎，將上述之 BCD 加法器邏輯函數以 **Package 與 component 語法包裝**，並以實驗板上兩顆七段顯示器驗證 BCD 加法電路是否正確。
2. 測試時，將以指撥開關輸入要相加的數字，而兩顆七段顯示器上必須可正確顯示出 **BCD 十進位加法運算**之結果。
3. 本次實驗，需使用A0~A7與B0~B7各表示以BCD編碼的兩個十進位數，其中A7~A4四個bit表示0~9(十位數)、A3~A0四個bit表示0~9(個位數)，B0~B3、B4~B7亦同，完成兩個十進位正整數的相加，結果亦為正整數，範圍為 0~99。
4. **禁用IF/SWITCH及Behavior model方式語法**，本次實驗需要以邏輯閘與階層式架構實作，使用其他方式則酌以扣分。



指定腳位

- 右表僅為範例
- 可自行選擇腳位

Name	Pin Location
A0	PIN_AB28 (SW[0])
A1	PIN_AC28 (SW[1])
A2	PIN_AC27 (SW[2])
A3	PIN_AD27 (SW[3])
A4	PIN_AB27 (SW[4])
A5	PIN_AC26 (SW[5])
A6	PIN_AD26 (SW[6])
A7	PIN_AB26 (SW[7])
B0	PIN_AC25 (SW[8])
B1	PIN_AB25 (SW[9])
B2	PIN_AC24 (SW[10])
B3	PIN_AB24 (SW[11])
B4	PIN_AB23 (SW[12])
B5	PIN_AA24 (SW[13])
B6	PIN_AA23 (SW[14])
B7	PIN_AA22 (SW[15])

Name	Pin Location
a0 (HEX0[0])	PIN_G18
b0 (HEX0[1])	PIN_F22
c0 (HEX0[2])	PIN_E17
d0 (HEX0[3])	PIN_L26
e0 (HEX0[4])	PIN_L25
f0 (HEX0[5])	PIN_J22
g0 (HEX0[6])	PIN_H22
a1 (HEX1[0])	PIN_M24
b1 (HEX1[1])	PIN_Y22
c1 (HEX1[2])	PIN_W21
d1 (HEX1[3])	PIN_W22
e1 (HEX1[4])	PIN_W25
f1 (HEX1[5])	PIN_U23
g1 (HEX1[6])	PIN_U24

Name	Pin Location
led (LEDR[0])	PIN_G19